SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

Analytical codes (10 QUESTIONS)

Scenario:

- Complex Scenario-Based Problem on Learning Algorithm, Loss Function, Chain Rule, and Gradient Descent
- A company is developing a deep learning model to predict the selling price of used cars.
- A single neuron is used for prediction with the following parameters:

* Input feature (car age): x = 4

* Actual selling price: y = 15

* Initial weight: W = 2

* Bias: b = 1

* Learning rate: \eta = 0.1

* Activation function: Identity, f(z)=z

* Loss function: Mean Squared Error (MSE)

The model is defined as:

\hat{y} = Wx + b

and the loss function is:

L=\frac{1}{2}(y-\hat{y})^2

Questions

1. Perform forward propagation and calculate the predicted output (\hat{y}).

```
# Given values
x = 4
W = 2
b = 1

# Forward Propagation
y_hat = W * x + b

print("Predicted Output =", y_hat)
```

```
Predicted Output = 9
```

2. Compute the loss using the MSE formula.

```
# Given values
y = 15
y_hat = 9

# Compute Loss (MSE)
loss = 0.5 * (y - y_hat) ** 2

print("Loss =", loss)
```

```
Loss = 18.0
```

+ Code + Text

3. Using the chain rule, derive the gradient \frac{\partial L}{\partial W}.

```
# Given values
x = 4
y = 15
y_hat = 9

# Gradient using Chain Rule
dL_dW = (y_hat - y) * x

print("Gradient dL/dW =", dL_dW)
```

```
Gradient dL/dW = -24
```

4. Calculate the updated weight after one iteration of gradient descent.

```
# Given values
W = 2
learning_rate = 0.1
dL_dW = -24

# Update weight
W_new = W - learning_rate * dL_dW

print("Updated Weight =", W_new)
```

```
Updated Weight = 4.4
```

5. Calculate the updated bias.

```python
# Given values
b = 1
learning_rate = 0.1
y = 15
y_hat = 9

# Gradient of bias
dL_db = y_hat - y

# Update bias
b_new = b - learning_rate * dL_db

print("Updated Bias =", b_new)
```

```
Updated Bias = 1.6
```

6. If the learning rate is increased from 0.1 to 1.0, discuss how it affects convergence and model stability.

```python
# Given values
W = 2
b = 1
learning_rate = 1.0

dL_dW = -24
dL_db = -6

# Updated parameters
W_new = W - learning_rate * dL_dW
b_new = b - learning_rate * dL_db

print("Updated Weight =", W_new)
print("Updated Bias =", b_new)
```

```
Updated Weight = 26.0
Updated Bias = 7.0
```

7. If the activation function is changed from the identity function to ReLU, explain whether the prediction and gradient computation will change for this input.

```
# Given values
x = 4
W = 2
b = 1

# Calculate z
z = W * x + b

# ReLU activation
if z > 0:
    y_hat = z
    derivative = 1
else:
    y_hat = 0
    derivative = 0

print("z =", z)
print("Prediction using ReLU =", y_hat)
print("ReLU Derivative =", derivative)
```

```
z = 9
Prediction using ReLU = 9
ReLU Derivative = 1
```

8. Suggest two methods to reduce the loss further without changing the dataset.

Method 1: Train for More Epochs

- Continue training the model for more iterations (epochs).
- This allows the weights and bias to update further, reducing the prediction error.
- As training continues, the loss gradually decreases until the model converges.

Method 2: Tune the Learning Rate

- Choose an appropriate learning rate.
- A very high learning rate may overshoot the minimum loss, while a very low learning rate makes training slow.
- Selecting an optimal learning rate helps the model converge faster and achieve a lower loss.

```
print("Methods to reduce the loss:")
print("1. Train the model for more epochs.")
print("2. Tune the learning rate.")
```

```
Methods to reduce the loss:
1. Train the model for more epochs.
2. Tune the learning rate.
```

9. Explain how backpropagation uses the chain rule to update parameters in a multilayer neural network.

```
print("Backpropagation Steps:")
print("1. Perform forward propagation.")
print("2. Calculate the loss.")
print("3. Use the chain rule to compute gradients.")
print("4. Update weights and bias using gradient descent.")
print("5. Repeat until the loss is minimized.")
```

```
Backpropagation Steps:
1. Perform forward propagation.
2. Calculate the loss.
3. Use the chain rule to compute gradients.
4. Update weights and bias using gradient descent.
5. Repeat until the loss is minimized.
```

10. If another feature (car mileage) is added, explain how the learning algorithm and weight updates will change.

```
# Two input features
age = 4
mileage = 60000

# Initial weights and bias
W1 = 2
W2 = 0.0001
b = 1

# Forward propagation
y_hat = W1 * age + W2 * mileage + b

print("Predicted Output =", y_hat)

print("\nWeight Update Equations:")
print("W1_new = W1 - learning_rate * dL/dW1")
print("W2_new = W2 - learning_rate * dL/dW2")
print("b_new = b - learning_rate * dL/db")
```

```
Predicted Output = 15.0

Weight Update Equations:
W1_new = W1 - learning_rate * dL/dW1
W2_new = W2 - learning_rate * dL/dW2
b_new = b - learning_rate * dL/db
```